

Interview Project Report - Phase Two

Orchid Pharmed Company

Ehsan Saleh

Task 1 - Part One

Initially, the dataset was examined regarding content, text, labels, and ratings. Given that the problem statement mentions a "Neutral" label, this label was not included in the original file. After evaluation, it appeared that texts where the Rating was given a value of 3 represent a middle ground and could be considered as "Neutral" (e.g., rows 37, 38, and 39). Furthermore, based on the Ratings, certain labels seemed to be incorrectly assigned; for instance, row 6 contains a positive comment but was labeled as 0. Consequently, in the first step, labeling was performed for the Neutral class, followed by classification incorporating the Neutral class in the subsequent stage.

Step 1: Neutral Labeling

Code for Step 1 - Task 1 - Part One (Google Colaboratory)

To label neutral texts, the VADER (Valence Aware Dictionary and Sentiment Reasoner) lexical sentiment analysis algorithm was utilized alongside a hybrid decision rule based on numerical scores and textual sentiment scores. First, patient feedback texts are extracted from the Feedback column, and using the `SentimentIntensityAnalyzer` tool from the NLTK library, the sentiment score for each text is calculated.

By relying on a predefined dictionary and linguistic rules (Positive Words and Negative Words), the VADER algorithm generates a compound score ranging between -1 and +1, indicating the degree of negativity or positivity of the text. Then, to determine the final sentiment label, a rule-based function was designed to simultaneously evaluate the textual compound score and the numerical rating submitted by the user (Ratings):

- If the numerical rating is low (≤ 2) or the textual score is highly negative, the text is labeled as **Negative (0)**.
- If the numerical rating is high (≥ 4) or the textual score is highly positive, it is labeled as **Positive (1)**.
- Otherwise, it is labeled as **Neutral (2)**.

Thus, neutral texts in this method are identified when there are neither strong positive nor strong negative indications in the text or numerical rating. In the final step, these text labels are converted into standard numerical values and saved into a new CSV file for subsequent analyses, such as topic modeling or LLM training.

As a result of this analysis, **646 comments** were categorized into the positive class, **304 comments** into the negative class, and **46 comments** into the neutral class.

Step 2: Classification Using LLM Model

Code for Step 2 - Task 1 - Part One (Google Colaboratory)

In this code, a deep learning model based on the Transformer architecture was designed and trained for sentiment analysis across three classes: negative, positive, and neutral/indifferent. Initially, required libraries such as `transformers`, `torch`, `datasets`, and `scikit-learn` are installed and loaded. Then, by checking GPU availability, it is ensured that adequate computational conditions are present for training heavier Transformer models, as these models run at extremely low speeds without GPU acceleration. Next, the data—comprising patient textual comments and sentiment labels—is loaded from Google Drive, with the text column selected as input and the numerical sentiment label as output. The data is split into Train, Valid, and Test sets using a stratified approach to preserve class proportions across splits and ensure a fair evaluation.

Subsequently, the data is converted into the standard Hugging Face `DatasetDict` format for direct integration into the training pipeline. Following this, the distribution of classes and text lengths is examined, revealing class imbalance and

sentence length diversity—factors influencing model architecture and loss function design (Figure 1 and Figure 2). To address this imbalance, class weighting is applied so that minority classes receive higher weight in the Loss function, making the model more sensitive to them.

In the next step, texts are converted into numerical tokens using the tokenizer of DistilBERT (a lightweight and fast variant of BERT that retains high accuracy with reduced computational overhead). The core algorithm consists of a DistilBERT-based Transformer model that leverages the Self-Attention mechanism to capture semantic relationships across the sentence, then calculates the probability of text belonging to each of the three sentiment classes through a classification layer.

To apply class weights, a custom `Trainer` is defined using weighted Cross-Entropy loss, ensuring minority class errors exert a stronger impact on learning. Training hyperparameters—such as learning rate, batch size, and epoch count—are configured with Early Stopping to halt training if validation performance stops improving, preventing overfitting. After training completion, the model is evaluated on the unseen test set using metrics like Accuracy, macro F1-score, and a classification report (Figure 3). Finally, the best model checkpoint is saved and deployed via a pipeline for predicting new text sentiments, demonstrating practical deployment readiness.

Although the number of neutral samples is significantly lower than the other two classes, the model exhibits a solid capability in identifying neutral sentences (Figure 4).

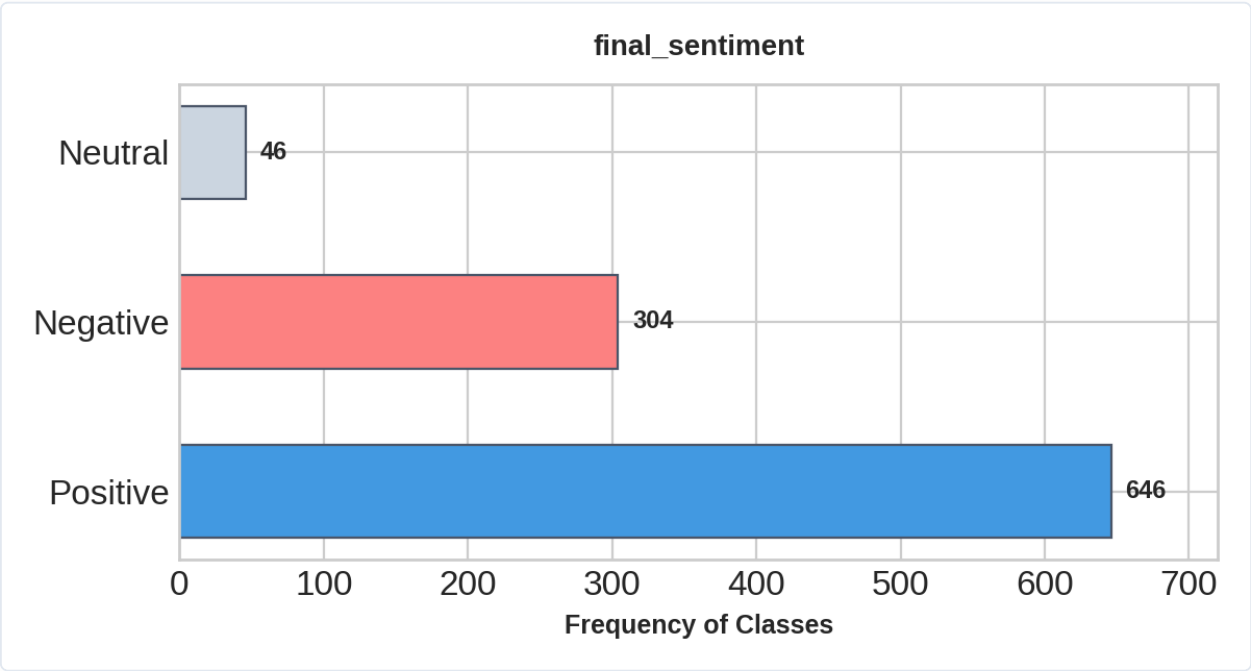


Figure 1 - Frequency of Classes

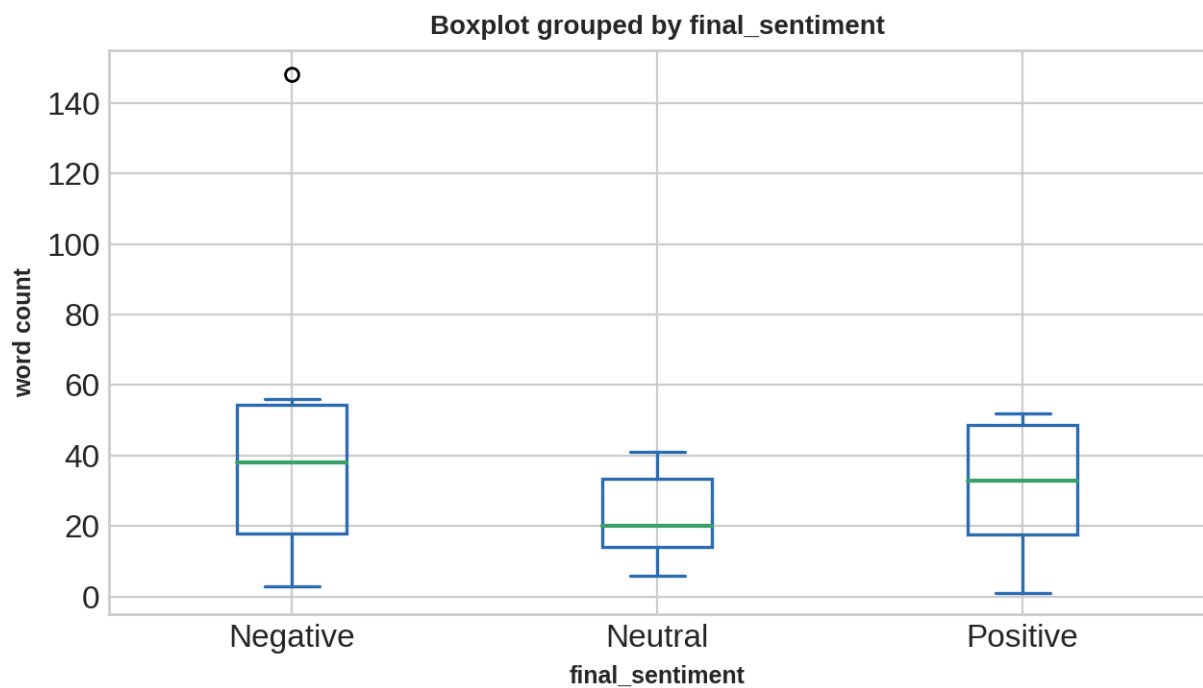


Figure 2 - Boxplot grouped by final_sentiment (word count)

```

--- FINAL TEST SET EVALUATION ---
Final Test Accuracy: 0.9100
Final Test F1 (Macro): 0.8020

Classification Report (Test Set):
      precision    recall  f1-score   support
 Negative         0.80      0.96      0.87         30
 Positive         0.95      0.97      0.96         65
   Neutral         0.44      0.57      0.80          5

 accuracy          0.91         100
 macro avg         0.80         0.79      0.86         100
 weighted avg      0.93         0.91      0.92         100

Model saved to /gdrive/MyDrive/OrchidPharmed/best_sentiment_model_testsplit

```

Figure 3 - Final Test Set Evaluation

```

Device set to use cuda:0
Text: 'there are good nurses but the environment was not clean'
Predicted Label: Neutral
Confidence Score: 0.6149

```

Figure 4 - Model Sample Prediction

It is worth noting that incorporating the neutral label into the dataset yields superior overall results compared to considering only the original two-class (positive and negative) dataset, as shown in Figure 5. This indicates that improved labeling was achieved.

Two-class code for original dataset

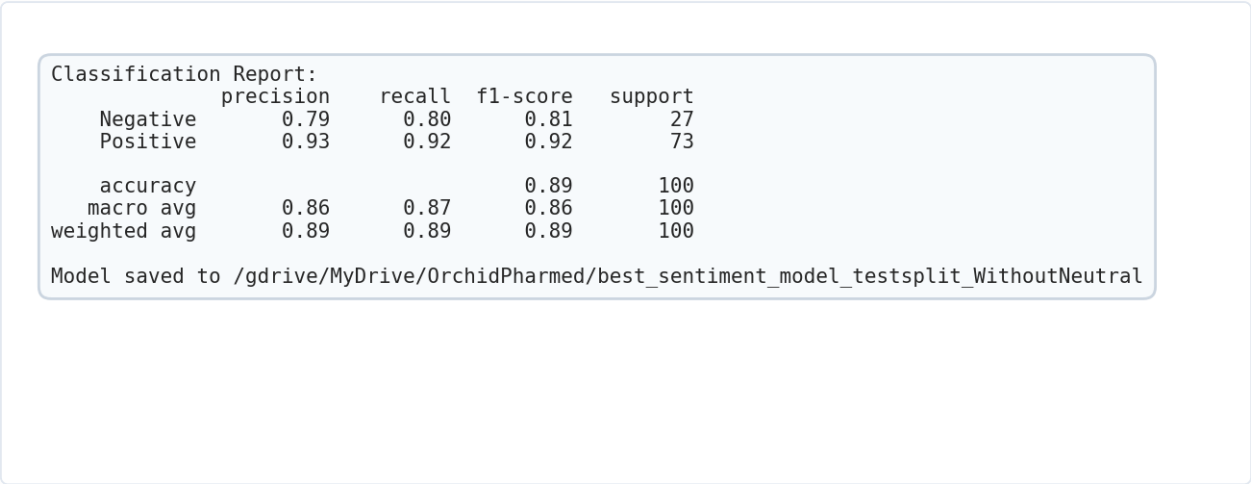


Figure 5 - Classification Report (Without Neutral)

Task 1 - Part Two

Code for Task 1 - Part Two

In this phase, meaningful insights are extracted from review text, including the identification of frequent keywords and high-level themes, facilitating a better understanding of patterns, core issues, and prominent trends within customer feedback.

The first section of the code aims to extract and analyze frequent keywords from user reviews to identify common language patterns in positive and negative feedback. First, a function named `get_top_ngrams` is defined, which utilizes N-gram extraction and `CountVectorizer` from `Scikit-learn` to extract multi-word expressions (here, bigrams or 2-word phrases) from text collections. In this process, English stop words are removed to maintain focus on meaningful vocabulary. Subsequently, phrase frequencies are calculated and returned in sorted order. A second function named `plot_ngrams` visually presents these results via horizontal bar charts for straightforward frequency comparison.

Next, data is split by sentiment label into negative and positive review subsets, and top 2-word phrases are extracted and plotted for each subset. This analysis clarifies which core concepts, problems, or strengths users emphasize in their negative and positive reviews, serving as an effective tool for identifying key issues, prioritizing improvements, and extracting actionable insights from textual data (Figure 6 and Figure 7).

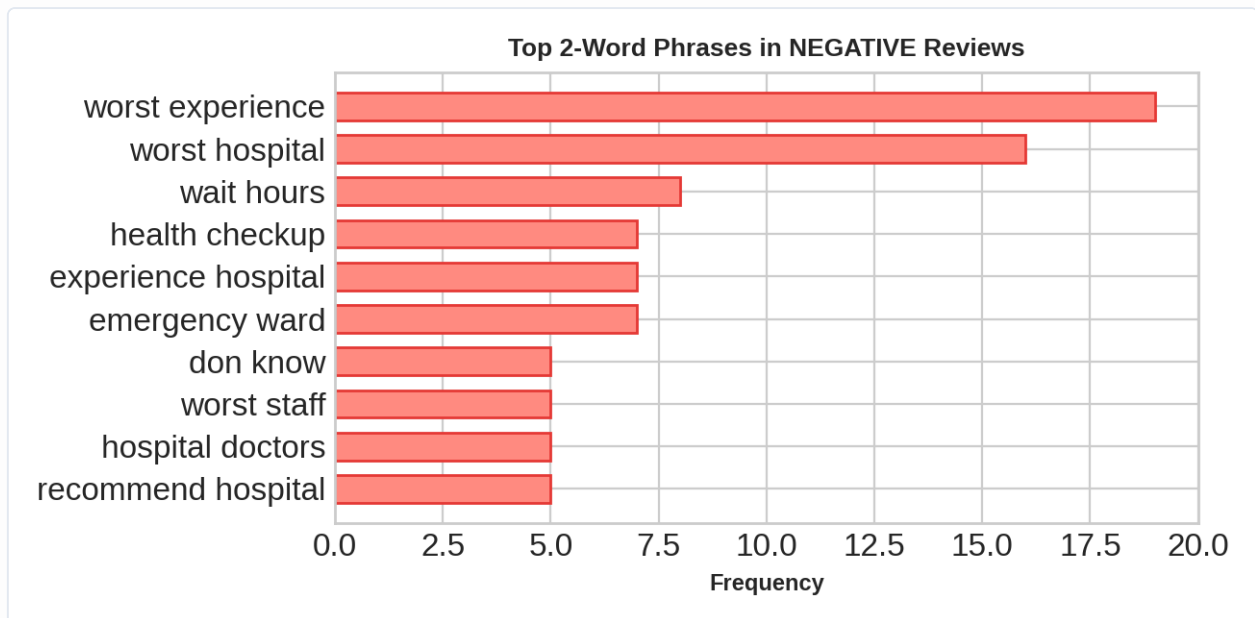


Figure 6 - Top 2-Word Phrases in NEGATIVE Reviews

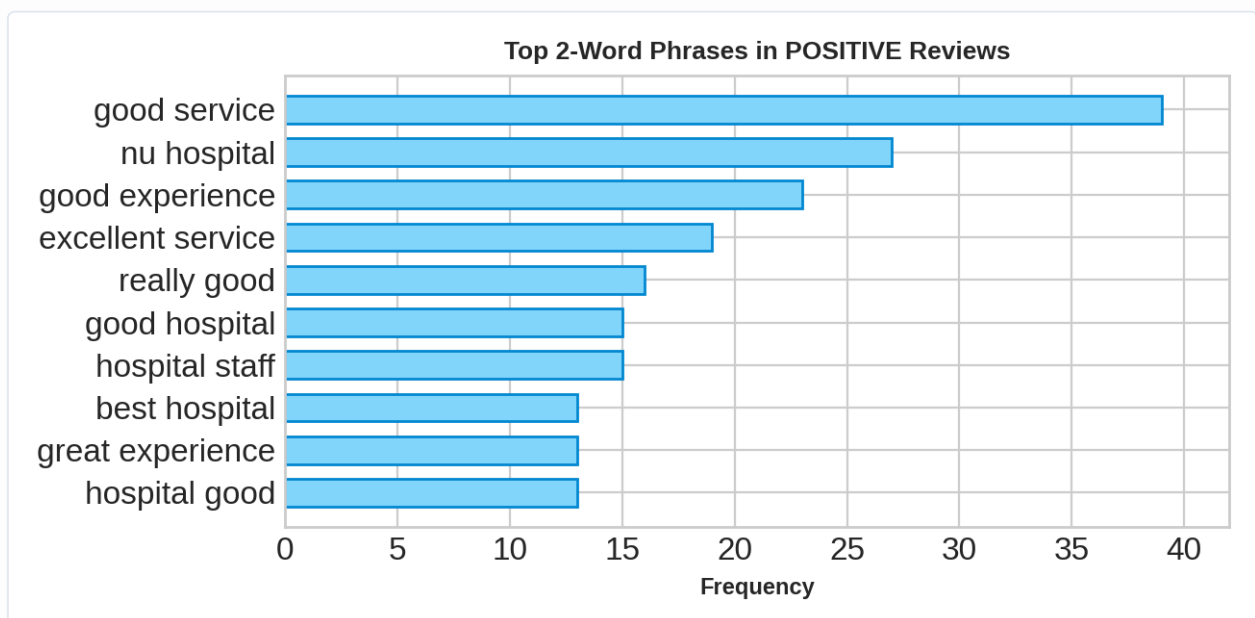


Figure 7 - Top 2-Word Phrases in POSITIVE Reviews

In the subsequent code block, topic modeling via BERTopic is implemented to extract High-Level Themes from user text reviews. Data is loaded from CSV, converting the feedback column to text format for NLP processing. A function named `analyze_specific_class` is defined to enable separate topic analysis per sentiment class (e.g., negative or positive) for targeted thematic discovery. If review count for a class falls below a predefined threshold, processing halts to prevent unreliable outputs.

To reduce randomness and ensure reproducibility, a UMAP model with a fixed `random_state` is defined for dimensionality reduction and improved text clustering. BERTopic is initialized with this fixed UMAP instance. BERTopic combines Transformer semantic embeddings, dimensionality reduction, and clustering to extract meaningful topics. Following model training per class, topic details are displayed and top themes are visualized as bar charts (Figure 8). This enables managerial decision-making, service enhancement, and strength/weakness identification.

Next, code is designed to analyze negative feedback by extracting precise problem descriptors using spaCy NLP to parse Adjective + Noun pairs (e.g., "dirty room" or "rude staff"). Frequency counts of top 15 pairs are plotted in a bar chart to

highlight urgent fix priorities. The same process is applied to positive feedback to identify praised service/product aspects (Figure 9 and Figure 10).

Furthermore, positive feedback is categorized into business aspects (doctors, nurses, staff behavior, wait time, cleanliness, facilities, treatment quality) using Zero-Shot Classification with Hugging Face's facebook/bart-large-mnli model without task-specific training. Feedback counts per category are visualized in a pie chart, accompanied by representative user quotes. This methodology is also applied to negative feedback to pinpoint key complaint dimensions (Figure 11 and Figure 12).

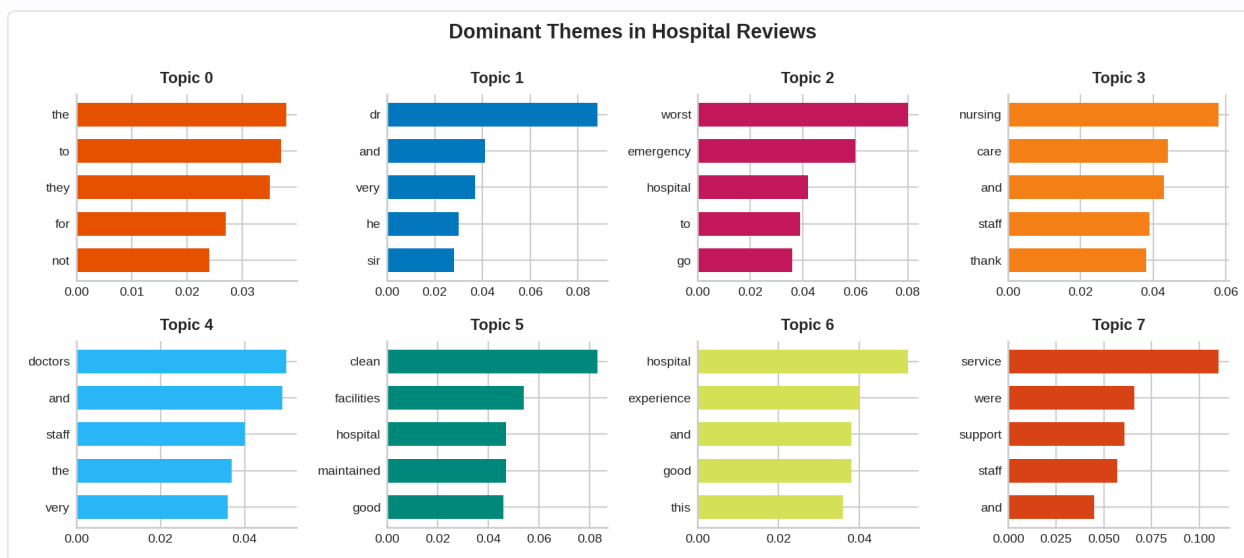


Figure 8 - Dominant Themes in Hospital Reviews

Figure 8 Detailed Explanation: Based on the BERTopic topic modeling phase, the displayed charts depict dominant themes extracted from patient reviews, where each Topic represents a semantic cluster sharing frequent vocabulary.

- **Topic 0:** Contains words like *the, to, they, for, not*, reflecting general sentence structures without a specific thematic focus.
- **Topic 1:** Includes words like *dr, and, very, he, sir*, referring to patient interactions with doctors and reflecting communication quality or respect.
- **Topic 2:** Displays words like *worst, emergency, hospital, to, go*, highlighting negative experiences particularly in emergency departments.
- **Topic 3:** Features words such as *nursing, care, and, staff, thank*, pointing to nursing care quality and staff appreciation (positive sentiment).
- **Topic 4:** Contains *doctors, and, staff, the, very*, focusing on overall medical staff cooperation and performance.
- **Topic 5:** Includes *clean, facilities, hospital, maintained, good*, directly relating to facility cleanliness and maintenance.
- **Topic 6:** Displays *hospital, experience, and, good, this*, reflecting overall patient experience and general satisfaction.
- **Topic 7:** Features *service, were, support, staff, and*, focusing on support service quality and staff behavior.

Overall, patient reviews center on healthcare service quality, physician/nurse conduct, emergency conditions, cleanliness, facilities, and overall visit experience, aiding in core institutional evaluations.

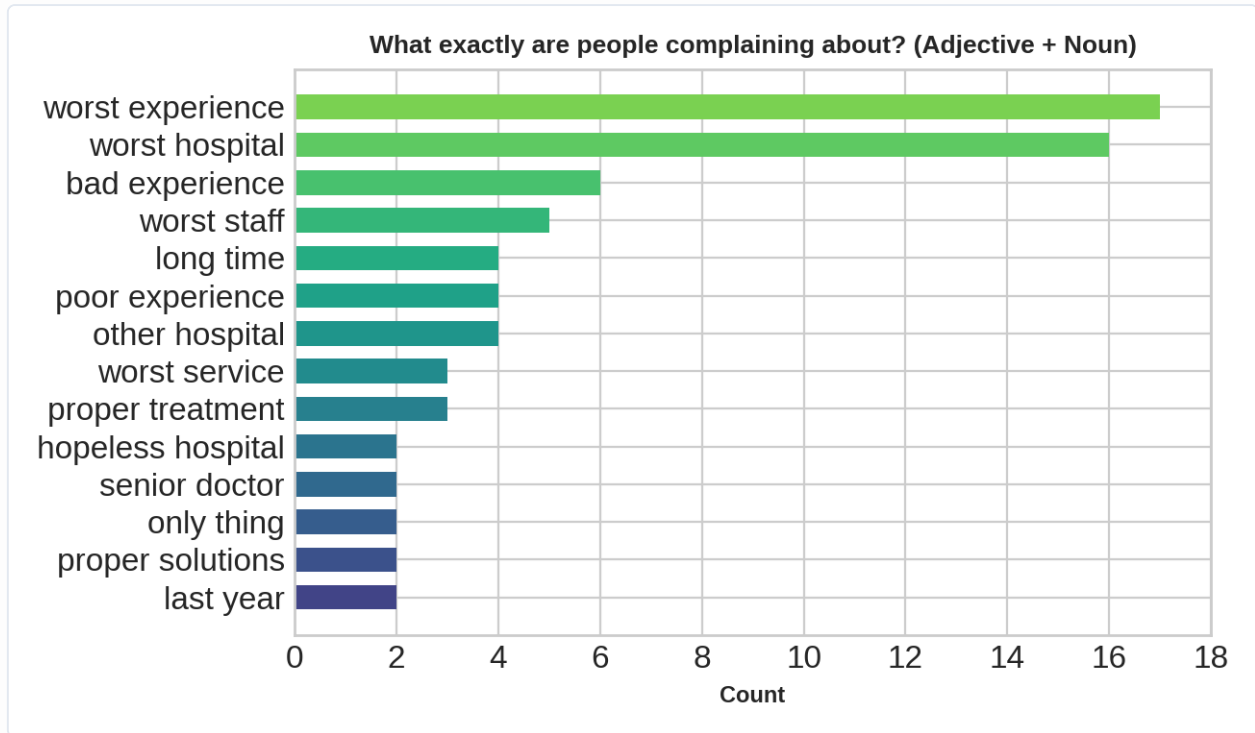


Figure 9 - What exactly are people complaining about? (Adjective + Noun)

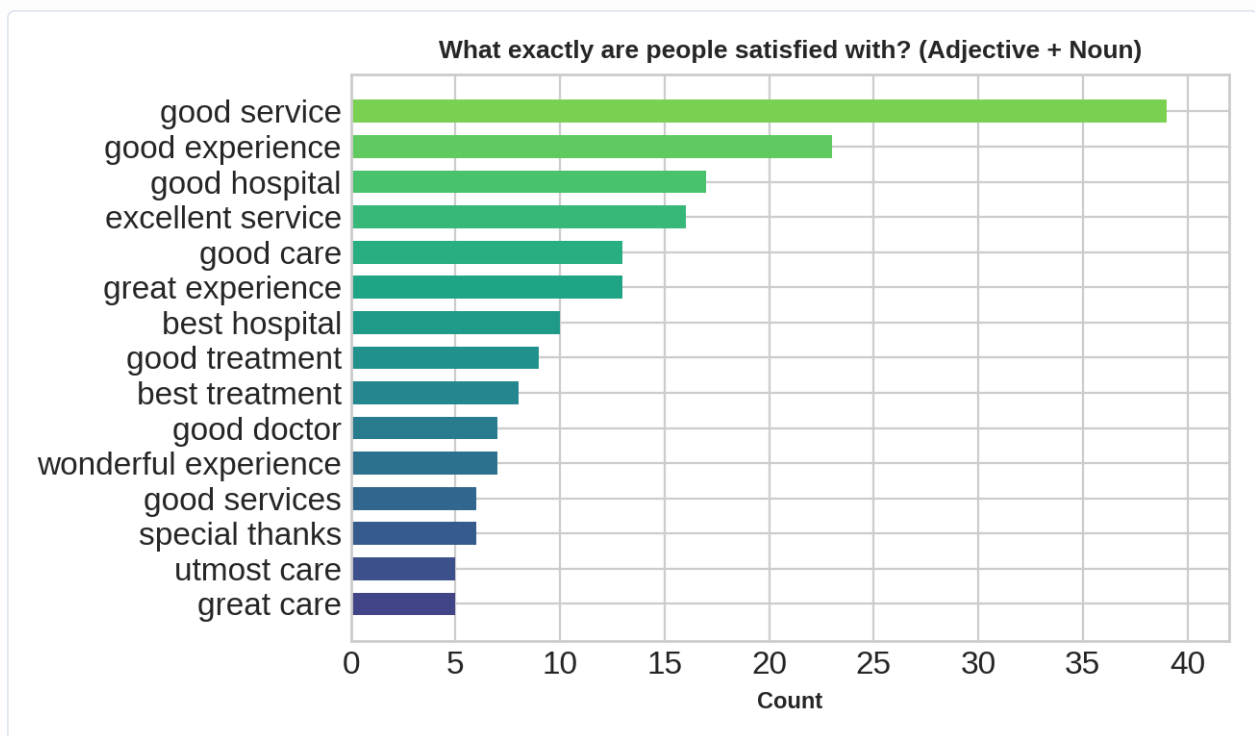


Figure 10 - What exactly are people satisfied with? (Adjective + Noun)

What do patients like the most? (Based on 646 reviews)

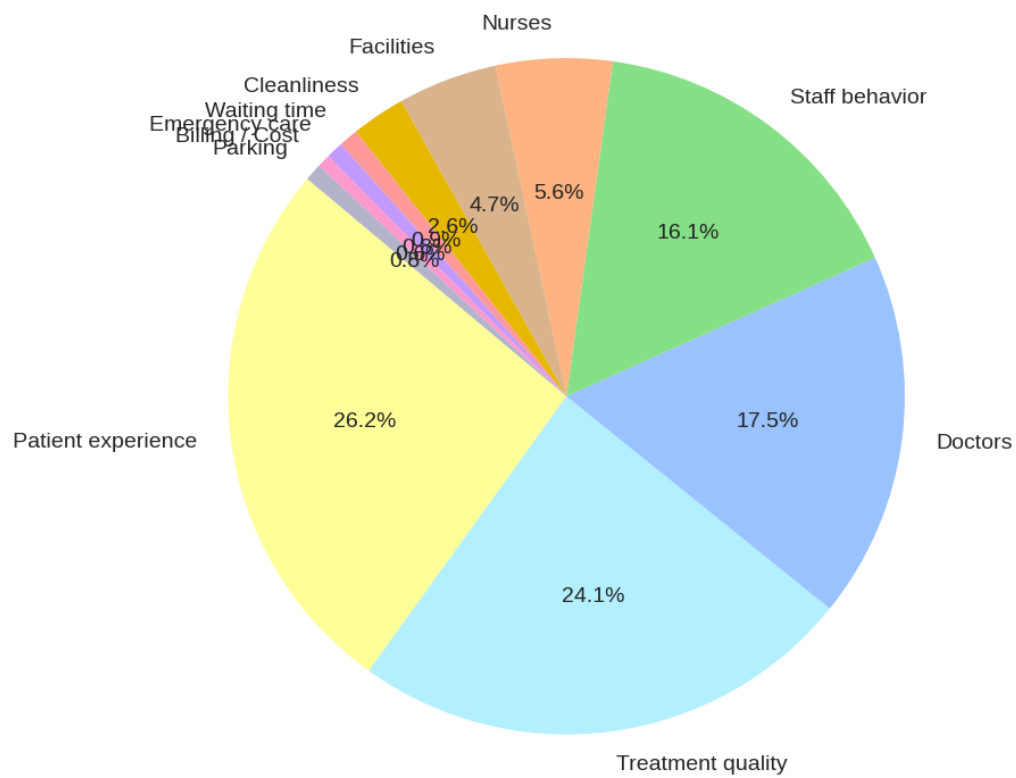


Figure 11 - What do patients like the most? (Based on 646 reviews)

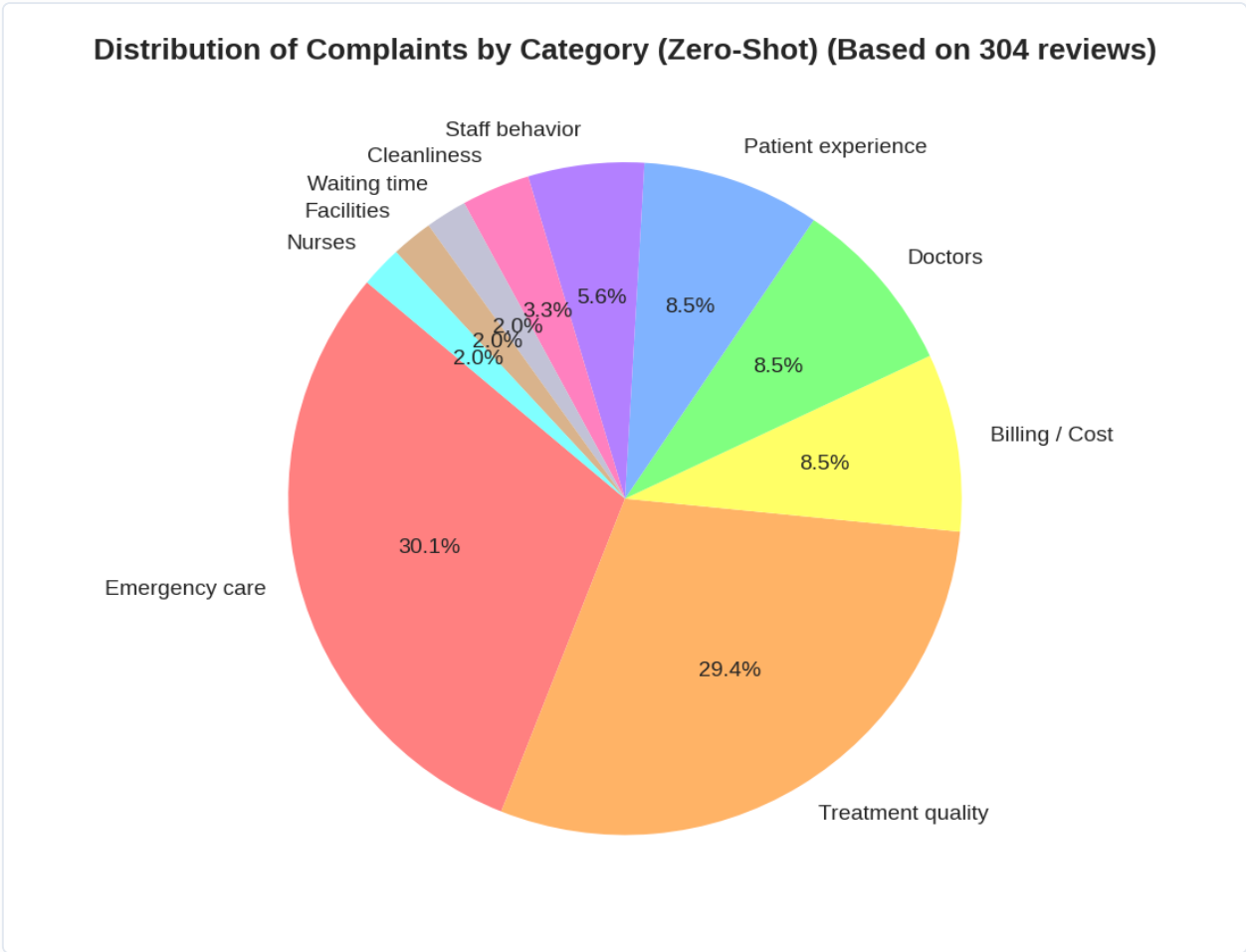


Figure 12 - Distribution of Complaints by Category (Zero-Shot) (Based on 304 reviews)

Task 1 - Part Two (RAG Question Answering System)

Code for Task 1 - Part Two (RAG)

In this section, an intelligent QA system based on PDF text using RAG (Retrieval-Augmented Generation) was implemented. First, a PDF file is read from Google Drive, extracted, and saved into a text file. The text is then segmented into small chunks so language models and search engines can process distinct sections efficiently.

Embeddings (vector representations) are generated for each chunk and stored in a FAISS vector database to enable fast similarity search. Using OpenAI's ChatOpenAI model (gpt-4o), accurate answers to user queries are generated based exclusively on retrieved context. When a query is submitted, relevant chunks are retrieved from FAISS, and the LLM produces a response based strictly on those chunks; if information is absent, the model responds with "I don't know".

For example, asking the paper-specific question: "What are cnn models used in this paper?" yields the correct precise answer:

"The CNN models used in this paper are Efficient Net-B0, Efficient Net-B1, Efficient Net-B2, Inception-v3, and Xception." (Figure 13).

```
1 # Finally, we can ask a question and get an answer.  
2 query = "What are cnn models used in this paper?"  
3 answer = qa_chain.invoke({"question": query})  
4 print(f"\n\nFinal answer:\n{answer}")
```

Final answer:

The CNN models used in this paper are Efficient Net-B0, Efficient Net-B1, Efficient Net-B2, In

Figure 13 - RAG System Query and Final Answer Execution Output